

APÊNDICE C – MÓDULO 3: APLICAÇÕES AVANÇADAS DE CONTROLE DIGITAL COM ARDUINO

C.1 Atividade 11: Controle de Mini Braço Robótico (2 Servos)

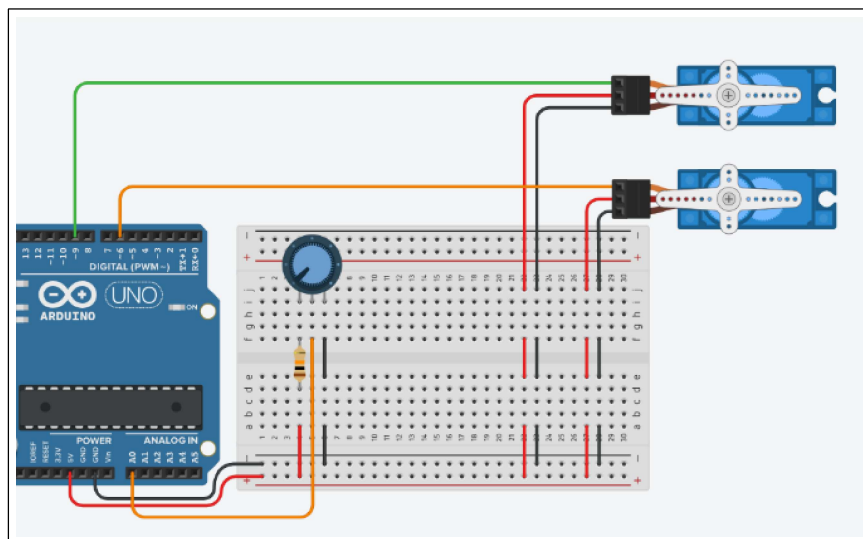
Conteúdo: Controle coordenado de servomotores e conversão analógica para ângulo.

Objetivo: Controlar dois servomotores simultaneamente para simular o movimento de um braço robótico básico, utilizando leitura analógica e mapeamento de valores no ambiente virtual *Tinkercad*.

Atividade Computacional – Etapas

1. Acesse a plataforma *Tinkercad* e crie um novo circuito.
2. Reproduza o esquema de ligação da Figura 20, contendo:
 - Servomotor 1 conectado ao pino digital D6;
 - Servomotor 2 conectado ao pino digital D9;
 - Potenciômetro conectado ao pino analógico A0.
3. Após revisar as conexões, programe o Arduino conforme as questões A, B e C.

Figura 20 – Dois servomotores controlados por um potenciômetro.



Fonte: Elaborada pelo autor.

Questão A – Movimento proporcional

Código:

Código-fonte 31 – Controle proporcional de dois servomotores.

```
1  #include <Servo.h>
2
3  Servo s1, s2;
4
5  void setup() {
6      s1.attach(6);
7      s2.attach(9);
8  }
9
10 void loop() {
11     int v = analogRead(A0);
12     int ang = map(v, 0, 1023, 0, 180);
13
14     s1.write(ang);
15     s2.write(180 - ang);
16 }
```

PREVER (Sim/Não)

- Os servomotores têm a mesma amplitude de movimentação?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe o comportamento dos servomotores em relação à posição do potenciômetro e explique como ocorre o movimento proporcional entre eles.

Questão B – Movimentos independentes

Altere o comportamento dos servomotores utilizando o código abaixo:

Código:

Código-fonte 32 – Controle com amplitudes diferentes para os servomotores.

```
1 s1.write(map(v,0,1023,30,150));
2 s2.write(map(v,0,1023,60,120));
```

PREVER (Sim/Não)

- Ao alterar os parâmetros da função `map()`, os servomotores permanecerão com a mesma amplitude de movimentação?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, explique como a alteração na função `map()` modificou o comportamento e a amplitude de movimentação dos servomotores.

Questão C – Movimento em etapas

Agora, modifique o código para que o servomotor atue em posições fixas utilizando estruturas condicionais.

Código:

Código-fonte 33 – Controle do servomotor em etapas utilizando condicionais.

```
1 if (v < 300)
2     s1.write(0);
3 else if (v < 600)
4     s1.write(90);
5 else
6     s1.write(180);
```

PREVER (Sim/Não)

- Com o uso das condicionais, os servomotores atuarão em etapas ao invés de forma contínua?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, explique como funciona a divisão das etapas de atuação dos servomotores e como as estruturas condicionais alteram o comportamento do movimento.

C.2 Atividade 12: Alarme com LDR e *Buzzer*

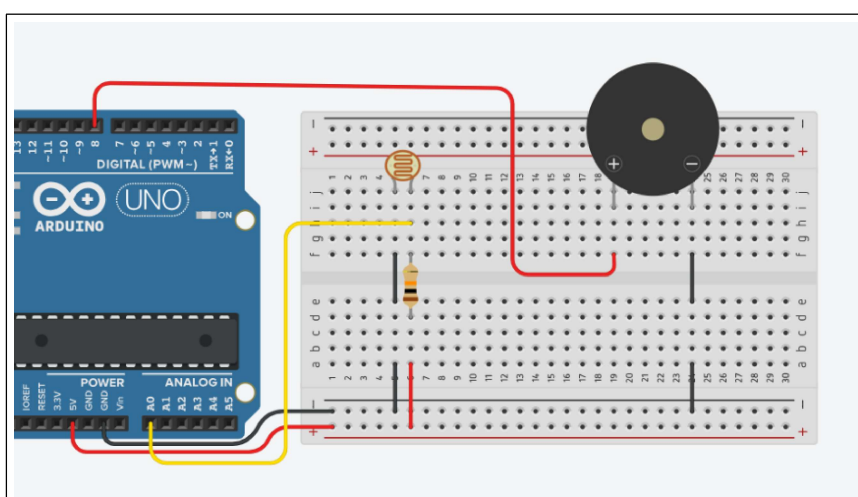
Conteúdo: Sensor de luminosidade, comparação de limiar e atuador sonoro.

Objetivo: Criar um alarme simples que ativa um *buzzer* quando o ambiente está escuro, utilizando leitura analógica e lógica condicional no ambiente virtual *Tinkercad*.

Atividade Computacional – Etapas

1. Acesse a plataforma *Tinkercad* e crie um novo circuito.
2. Reproduza o esquema de ligação da Figura 21, contendo:
 - LDR e resistor de $10k\Omega$ formando um divisor resistivo ligado ao pino analógico A0;
 - *Buzzer* ativo conectado ao pino digital D8.
3. Após revisar as conexões, programe o Arduino conforme as questões A, B e C.

Figura 21 – LDR com *buzzer*.



Fonte: Elaborada pelo autor.

Questão A – Leitura do sensor

Código:

Código-fonte 34 – Leitura analógica do sensor LDR no monitor serial.

```
1 void setup() {  
2     Serial.begin(9600);  
3 }  
4  
5 void loop() {  
6     int v = analogRead(A0);  
7     Serial.println(v);  
8     delay(300);  
9 }
```

PREVER (Sim/Não)

- Com esse código implementado, o sensor LDR tem alguma atuação além de imprimir o valor capturado no monitor serial?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe e explique como se comporta o valor apresentado no monitor serial em relação à luminosidade. Comente também sobre o intervalo registrado pelo sensor e o intervalo impresso no monitor serial.

Questão B – Alarme por limiar

Código:

Código-fonte 35 – Acionamento do buzzer por limiar de luminosidade.

```
1 void setup() {  
2     pinMode(8, OUTPUT);  
3 }
```

```

4
5 void loop() {
6     int v = analogRead(A0);
7
8     if (v < 150)
9         digitalWrite(8, HIGH);
10    else
11        digitalWrite(8, LOW);
12 }

```

PREVER (Sim/Não)

- O alarme soa a partir de um intervalo de luminosidade?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe e determine qual o valor mínimo que deve ser capturado para acionar o alarme. Explique como a comparação com o limiar definido no código influencia o acionamento do *buzzer*.

Questão C – Alarme intermitente

Código:

Código-fonte 36 – Acionamento intermitente do buzzer por limiar de luminosidade.

```

1 void setup() {
2     pinMode(8, OUTPUT);
3 }
4
5 void loop() {
6     int v = analogRead(A0);
7
8     if (v < 350) {
9         digitalWrite(8, HIGH);

```

```

10     delay(200);
11     digitalWrite(8, LOW);
12     delay(200);
13 } else {
14     digitalWrite(8, LOW);
15 }
16 }

```

PREVER (Sim/Não)

- O alarme agora tem o mesmo comportamento de quando implementado com o código da questão anterior?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe e determine o novo comportamento do alarme. Explique também como as alterações no código, especialmente o uso dos comandos `delay()`, influenciaram esse novo comportamento.

C.3 Atividade 13: Porta Automática com Sensor Ultrassônico e Servomotor

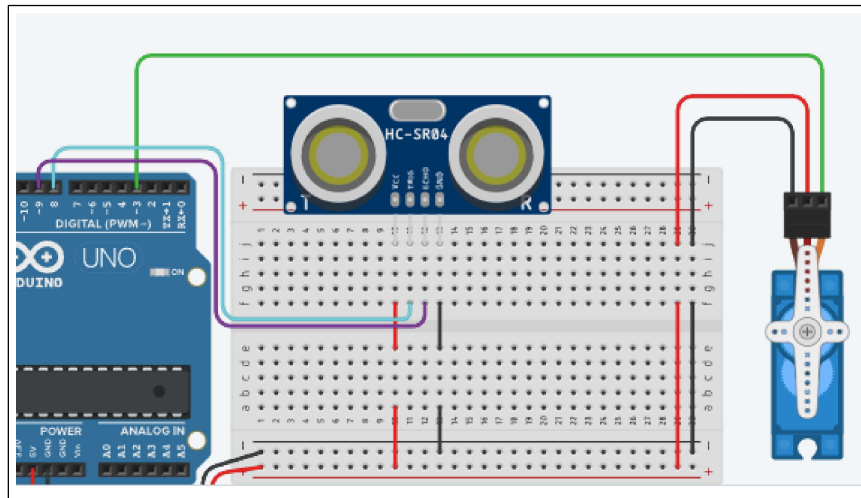
Conteúdo: Leitura de sensor ultrassônico, comparação por distância e acionamento de servomotor.

Objetivo: Detectar a aproximação de um objeto ou pessoa usando o sensor HC-SR04, acionando automaticamente um servomotor que simula a abertura de uma porta.

Atividade Computacional – Etapas

1. Acesse a plataforma *Tinkercad* e crie um novo circuito.
2. Reproduza o esquema de ligação da Figura 22, contendo:
 - Sensor ultrassônico HC-SR04;
 - Pino Trigger do sensor conectado ao pino digital D8;
 - Pino Echo do sensor conectado ao pino digital D9;
 - Servomotor com sinal conectado ao pino digital D3.
3. Após revisar as conexões, programe o Arduino conforme as questões A, B e C.

Figura 22 – Porta automática com sensor ultrassônico e servomotor.



Fonte: Elaborada pelo autor.

Questão A – Leitura da distância

Código:

Código-fonte 37 – Leitura de distância com sensor ultrassônico HC-SR04.

```

1  #define TRIG 8
2  #define ECHO 9
3
4  void setup() {
5      Serial.begin(9600);
6      pinMode(TRIG, OUTPUT);
7      pinMode(ECHO, INPUT);
8  }
9
10 long medirDistancia() {
11     digitalWrite(TRIG, LOW);
12     delayMicroseconds(2);
13     digitalWrite(TRIG, HIGH);
14     delayMicroseconds(10);
15     digitalWrite(TRIG, LOW);
16

```



```

17   long duracao = pulseIn(ECHO, HIGH);
18   long distancia = duracao / 58;
19   return distancia;
20 }
21
22 void loop() {
23     long d = medirDistancia();
24     Serial.println(d);
25     delay(300);
26 }

```

PREVER (Sim/Não)

- Com esse código implementado, o sensor de distância consegue influenciar o comportamento do servomotor?

() Sim

() Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe os valores apresentados no monitor serial e determine os limites percebidos pelo sensor de distância. Explique também por que, neste código, o servomotor ainda não é acionado.

Questão B – Porta automática simples

Quando a distância medida for menor que 120 *cm*, o servomotor deverá abrir a porta.

Código:

Código-fonte 38 – Acionamento simples de porta automática por distância.

```

1  #include <Servo.h>
2
3  #define TRIG 8
4  #define ECHO 9
5

```

```
6 Servo porta;
7
8 long medirDistancia() {
9     digitalWrite(TRIG, LOW);
10    delayMicroseconds(2);
11    digitalWrite(TRIG, HIGH);
12    delayMicroseconds(10);
13    digitalWrite(TRIG, LOW);
14
15    long duracao = pulseIn(ECHO, HIGH);
16    return duracao / 58;
17 }
18
19 void setup() {
20     porta.attach(3);
21     porta.write(0);
22     pinMode(TRIG, OUTPUT);
23     pinMode(ECHO, INPUT);
24 }
25
26 void loop() {
27     long d = medirDistancia();
28
29     if (d < 120)
30         porta.write(90);
31     else
32         porta.write(0);
33 }
```

PREVER (Sim/Não)

- O servomotor se move com base na distância capturada pelo sensor ultrassônico?
- () Sim
- () Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe e explique qual intervalo de distância determina o estado da porta, indicando quando ela permanece aberta ou fechada pelo servomotor.

Questão C – Três níveis de ação do servomotor

Nesta etapa, o comportamento da porta será dividido em três faixas de distância:

- Distância menor que 50 *cm*: porta totalmente aberta;
- Distância entre 50 *cm* e 150 *cm*: porta meio aberta;
- Distância maior que 150 *cm*: porta fechada.

Código:**Código-fonte 39 – Controle do servomotor em três níveis de abertura.**

```
1  #include <Servo.h>
2
3  #define TRIG 8
4  #define ECHO 9
5
6  Servo porta;
7
8  long medirDistancia() {
9      digitalWrite(TRIG, LOW);
10     delayMicroseconds(2);
11     digitalWrite(TRIG, HIGH);
12     delayMicroseconds(10);
13     digitalWrite(TRIG, LOW);
14
15     long duracao = pulseIn(ECHO, HIGH);
16     return duracao / 58;
17 }
18
19 void setup() {
```

```
20  porta.attach(3);
21  porta.write(0);
22  pinMode(TRIG, OUTPUT);
23  pinMode(ECHO, INPUT);
24  }
25
26  void loop() {
27      long d = medirDistancia();
28
29      if (d < 50) {
30          porta.write(90);
31      }
32      else if (d <= 150) {
33          porta.write(45);
34      }
35      else {
36          porta.write(0);
37      }
38
39      delay(100);
40  }
```

PREVER (Sim/Não)

- Com esse código implementado, o servomotor passa a atuar por etapas?
- () Sim
- () Não

OBSERVAR E EXPLICAR

Após executar a simulação, observe e explique o novo comportamento da porta. Determine também quais são as etapas de atuação do servomotor e como cada faixa de distância influencia sua posição.